UNIVERSIDAD TECNOLÓGICA DE SAN JUAN DEL RÍO

Construcción paso a paso de una PWA

DECIMO CUATRIMESTRE

ING. EN TECNOLOGÍAS DE LA INFORMACIÓN, ÁREA DE DESARROLLO DE SOFTWARE MULTIPLATAFORMA. UNIDAD ACADEMICA JALPAN

NOMBRE DE LA MATERIA:

APLICACIONES WEB PROGRESIVAS

PRESENTA:

T.S.U. JOSE MANUEL VILLA AGUILLÓN
T.S.U. ARIEL GUEVARA BALDERAS
T.S.U. JULIAN DAVID SIERRA

PROFESOR DE ASIGNATURA:

ING. Daniel Martínez Trejo

JALPAN DE SERRA, QRO
04 DE NOVIEMBRE 2026

SECRETARÍA DE EDUCACIÓN QUERÉTARO · QUERÉTARO GOBIERNO DEL ESTADO Juntos, Adelante. · CONTIGO TODOS AVANZAMOS

RutaExpress

Aplicación Web Progresiva para el transporte local de Jalpan de Serra



Índice

Si el índice aparece vacío, en Word haz clic derecho sobre él y elige «Actualizar campos».

1. Introducción

RutaExpress es una Aplicación Web Progresiva (PWA) que funciona como guía del transporte público local de Jalpan de Serra, en la Sierra Gorda de Querétaro. Reúne en un solo lugar las paradas de autobús y las bases de combis, las rutas que pasan por cada una, los horarios de primera y última salida, las tarifas, los comentarios de los usuarios y los avisos de desvíos o retrasos.

En la sierra la señal de celular es intermitente, por eso el funcionamiento sin conexión no es un adorno: la aplicación abre, muestra el catálogo y guarda los comentarios aunque no haya internet, y los envía sola cuando la red regresa.

1.1 Objetivo

Construir paso a paso una Aplicación Web Progresiva completa, desde el entorno de desarrollo hasta su publicación, aplicando app shell, manifiesto, service worker, almacenamiento local, funcionamiento offline, sincronización en segundo plano y notificaciones push. Al terminar cada fase se verificó su funcionamiento y se registró un commit en el repositorio.

1.2 Resultados esperados y estado

Resultado esperado	Estado	Evidencia
La PWA se instala	Cumplido	Edge reporta 0 errores de instalabilidad en la URL publicada; botón «Instalar app» con instrucciones de respaldo
Funciona sin conexión	Cumplido	Catálogo, fichas y mapa visitado disponibles con el servidor apagado (sección 7)
Sincroniza reseñas al recuperar la red	Cumplido	Background Sync envía la cola de IndexedDB a Supabase; probado offline → online
Recibe notificaciones push	Cumplido	Suscripción real con VAPID y entrega a través del servicio push de Windows (WNS)
Repositorio con al menos un commit por fase	Cumplido	18 commits: uno por cada fase 0 a 13 más los de las mejoras (Anexo B)
Aplicación publicada en GitHub Pages	Cumplido	https://kaidav54.github.io/pwaauto/ con despliegue automático

2. Funcionalidades

La aplicación tiene cinco secciones accesibles desde el menú inferior: Inicio, Paradas, Cerca, Avisos y Ajustes.

Función	Qué hace	Tecnología principal
Catálogo de paradas (CSR)	Lista con buscador por nombre, zona o ruta y filtros Autobús / Combis; el mapa se sincroniza con los filtros	Enrutador por hash, fetch, Leaflet
Ficha de parada (SSR)	Rutas, horario de primera y última salida, frecuencia y tarifa en un tablero estilo andén	Plantilla en el servidor + hidratación
Mapa real	Pines por parada en el catálogo, ubicación en la ficha y ruta a pie en «Cerca de mí»	Leaflet 1.9.4 + OpenStreetMap
Cerca de mí	Encuentra la parada más cercana con distancia, rumbo y minutos caminando	Geolocation API, fórmula de Haversine
Reseñas en tiempo real	Comentarios de usuarios que llegan al instante a otros celulares; botón para reportar	Supabase (PostgreSQL + Realtime)
Reseñas sin conexión	El comentario se guarda en el celular y se envía solo al volver la señal	IndexedDB + Background Sync
Avisos push	Desvíos, obras y retrasos aunque la app esté cerrada	Push API, VAPID, web-push
Favoritas y última consultada	Accesos rápidos en el Inicio	IndexedDB, localStorage
Instalación	Ícono en el teléfono, ventana propia y splash	Manifiesto, beforeinstallprompt
Actualizaciones	Aviso «Hay una nueva versión» con botón Actualizar	Ciclo de vida del service worker
Tema claro / oscuro	Según el sistema o elegido en Ajustes	CSS custom properties, localStorage
Compartir y vibrar	Hoja nativa para compartir una parada; vibración al confirmar acciones	Web Share API, Vibration API

3. Tecnologías utilizadas

Capa	Tecnología	Uso en el proyecto
Servidor	Node.js 24 + Express 4.22	Archivos estáticos, API del catálogo, ficha SSR, API de reseñas de respaldo y envío de push
Push	web-push 3.6 (VAPID)	Firmar y enviar notificaciones a los navegadores suscritos
Base de datos	Supabase (PostgreSQL)	Reseñas compartidas con reglas Row Level Security, antispam y reportes
Tiempo real	Supabase Realtime (supabase-js 2.117.3)	Avisar a los celulares cuando llega una reseña nueva
Mapas	Leaflet 1.9.4 + OpenStreetMap	Mapas interactivos; Leaflet se sirve desde la propia app para funcionar offline
Navegador	Service Worker, Cache Storage, IndexedDB, localStorage	Caché del shell, datos sin conexión, cola de envíos y preferencias
APIs del dispositivo	Geolocation, Notifications, Push, Background Sync, Vibration, Web Share	Funciones físicas y en segundo plano
Interfaz	HTML, CSS y JavaScript (módulos ES) sin frameworks	App shell, vistas y componentes
Diseño	Bricolage Grotesque y Atkinson Hyperlegible (OFL), íconos SVG propios	Identidad visual de la Sierra Gorda
Imágenes	sharp	Convertir fotos a WebP optimizado en varios tamaños
Pruebas	node:test + Edge con Chrome DevTools Protocol	10 pruebas automáticas del servidor y pruebas de extremo a extremo
Publicación	Git, GitHub, GitHub Actions, GitHub Pages	Control de versiones y despliegue automático en cada push

4. Arquitectura

4.1 Vista general

El proyecto tiene tres partes que se comunican por HTTP:

- **Cliente PWA (carpeta public/).** Es lo único que se publica en GitHub Pages: shell, vistas, service worker, manifiesto, datos del catálogo, imágenes y fuentes.
- **Servidor Node (server.js).** Genera la ficha SSR, expone la API y envía las notificaciones push. Se usa en local con npm start.
- **Servicios externos.** Supabase guarda las reseñas, OpenStreetMap entrega los mapas y el servicio push del navegador (FCM o WNS) entrega las notificaciones.

La aplicación detecta si hay servidor consultando /api/salud y se adapta: con servidor usa la ficha SSR y el push; en GitHub Pages usa la ficha generada en el navegador. Todas las rutas son relativas para funcionar igual en «/» y en «/pwaauto/».

4.2 Estructura de carpetas

Ruta	Contenido
server.js	Servidor Express: API, SSR, reseñas y push
server/render.js · push.js · almacen.js	Plantilla de la página SSR, VAPID y suscripciones, persistencia en JSON
public/index.html	App shell
public/sw.js	Service worker: caché, offline, Background Sync, push y actualizaciones
public/manifest.json · offline.html	Manifiesto y página sin conexión
public/js/	22 módulos: enrutador, vistas, API, IndexedDB, reseñas, mapa, dispositivo, notificaciones…
public/data/paradas.json	Catálogo de paradas
public/img · fonts · icons · vendor	Fotos WebP y sprite SVG, tipografías, íconos de la app, Leaflet y supabase-js
scripts/	Generadores de íconos, claves VAPID e imágenes optimizadas
test/servidor.test.js	Pruebas automáticas
docs/	Reporte de pruebas, script SQL de Supabase y este documento
.github/workflows/pages.yml	Despliegue automático en GitHub Pages

4.3 Renderizado en el cliente (CSR) y en el servidor (SSR)

CSR: el catálogo, «Cerca de mí», Avisos y Ajustes se dibujan en el navegador. Un enrutador basado en el hash (#/paradas, #/parada/id) cambia de vista sin recargar la página y funciona en GitHub Pages sin configurar el servidor.

SSR: la ficha de cada parada (/parada/:id) llega del servidor como HTML completo, con las rutas, los horarios y las reseñas ya incluidos. Se ve aunque JavaScript esté desactivado y es mejor para buscadores. Después, el módulo ficha-ssr.js la «hidrata»: reutiliza ese HTML y le conecta los favoritos, el formulario de reseñas, el mapa y el botón de compartir.

Para no duplicar código, la plantilla de la ficha (plantilla-ficha.js) es un script compartido que usan tanto el servidor como el navegador.

4.4 Flujo de una reseña

1. El usuario escribe un comentario en la ficha de una parada.
2. La reseña se guarda primero en IndexedDB (almacén «pendientes») con un identificador único generado en el celular.
3. Se registra un Background Sync con la etiqueta «sync-resenas».
4. Cuando hay red, el service worker envía la reseña a Supabase. Si falla, el navegador lo reintenta más tarde. Gracias al identificador, un reintento no la duplica.
5. Supabase la guarda aplicando las reglas de seguridad y el límite antispam.
6. Supabase Realtime avisa a los demás celulares que tienen abierta esa parada y el comentario aparece sin recargar.

En navegadores sin Background Sync (Firefox y Safari) la página hace el envío por su cuenta al recuperar la conexión.

4.5 Estrategias de caché del service worker

Recurso	Estrategia	Motivo
App shell: HTML, CSS, JS, íconos, fuentes, Leaflet	Cache First (precache en install)	La app abre al instante y sin conexión
Datos: /api/* y data/paradas.json	Network First	Siempre lo más reciente; si no hay red, la última copia
Fichas SSR /parada/:id	Network First → copia en caché → ficha CSR	Una ficha nunca visitada se dibuja con los datos de IndexedDB
Imágenes	Stale-While-Revalidate	Se muestra rápido y se actualiza en segundo plano
Mapas (teselas de OpenStreetMap)	Cache First con límite de 400	Las zonas visitadas se ven sin conexión sin llenar el teléfono

4.6 Modelo de datos

Catálogo (public/data/paradas.json)

Cada parada tiene: id, nombre, tipo (autobus | combi), direccion, zona, lat, lng, iluminada, techada, accesible, descripcion y una lista de rutas con ruta, nombre, primera, ultima, frecuencia, tarifa.

Reseñas (tabla resenas en Supabase)

Columna	Tipo	Regla
id	uuid (llave primaria)	Lo genera el celular; evita duplicados en los reintentos
parada_id	text	Solo minúsculas, números y guiones
autor	text	1 a 40 caracteres; «Anónimo» por defecto
texto	text	3 a 500 caracteres
fecha / recibida	timestamptz	Cuándo se escribió (puede ser offline) y cuándo llegó
reportes / oculta	integer / boolean	Con 3 reportes la reseña se oculta sola

Seguridad (Row Level Security): cualquiera puede leer las reseñas visibles y publicar una nueva, pero nadie puede editar ni borrar desde la app. Un trigger limita a 8 comentarios por minuto en cada parada. El script completo está en docs/supabase.sql.

Almacenamiento local

- **IndexedDB «rutaexpress»:** paradas (respaldo del catálogo), resenas (copia local), pendientes (cola de envío), favoritos y meta (fecha de actualización y avisos recibidos).
- **localStorage:** tema, nombre para los comentarios y última parada consultada.
- **Cache Storage:** shell, datos, imágenes y mapas, separados por versión.

5. Desarrollo por fases

Cada fase se desarrolló en orden, se verificó (con pruebas automáticas o en el navegador Edge controlado por script) y se registró en un commit.

Fase 0 · Entorno de desarrollo y repositorio

Se verificaron Node.js 24, npm 11 y Git 2.55; se inicializó el repositorio, se instalaron Express y web-push y se creó un servidor mínimo con el endpoint /api/salud.

- **Archivos:** package.json, .gitignore, server.js
- **Verificación:** GET /api/salud responde {"ok":true}
- **Commit:** 35e0061

Fase 1 · App shell

Estructura fija de la app: cabecera, área de contenido y menú inferior de cinco secciones, con hoja de estilos responsiva y tema claro/oscuro.

- **Archivos:** public/index.html, public/css/styles.css
- **Verificación:** El shell se sirve y se ve en un viewport de celular
- **Commit:** eaa0e32

Fase 2 · Manifiesto e íconos

Un script propio genera los íconos PNG (normales, maskable, Apple y badge) sin dependencias externas. El manifiesto define nombre, colores, modo standalone, íconos y accesos directos.

- **Archivos:** scripts/generar-iconos.js, public/manifest.json, public/icons/
- **Verificación:** Manifiesto válido sin errores; íconos revisados visualmente
- **Commit:** 56eb2dd

Fase 3 · Service worker básico

Registro del service worker con rutas relativas y sw.js con los tres eventos base: install (precache), activate (limpieza de versiones) y fetch.

- **Archivos:** public/js/registro-sw.js, public/sw.js
- **Verificación:** Estado «activated» en el navegador
- **Commit:** 0f8daf5

Fase 4 · Renderizado CSR y SSR

API del catálogo (/api/paradas), enrutador por hash con las vistas del catálogo y la ficha, y ficha generada en el servidor con los datos incrustados para hidratar.

- **Archivos:** server.js, server/render.js, public/js/router.js, vistas.js, api.js, plantilla-ficha.js

- **Verificación:** 8 paradas listadas; /parada/:id devuelve HTML completo y 404 si no existe
- **Commit:** c350c81

Fase 5 · Splash y Home

Splash del sistema a partir del manifiesto, splash animado dentro de la app y Home con botón de instalación basado en beforeinstallprompt.

- **Archivos:** public/js/splash.js, instalacion.js
- **Verificación:** Splash y Home verificados en Edge
- **Commit:** 60ef834

Fase 6 · Datos locales

Capa de IndexedDB compartida entre la página y el service worker, preferencias en localStorage, favoritas y respaldo local del catálogo remoto. Vista de Ajustes.

- **Archivos:** public/js/db.js, prefs.js, ficha-acciones.js
- **Verificación:** IndexedDB con 8 paradas; el tema se conserva al recargar
- **Commit:** 4b5e2bc

Fase 7 · Caché y offline

Página offline.html, tres estrategias de caché (Cache First, Network First, Stale-While-Revalidate) e indicador «En línea / Sin conexión» en la cabecera.

- **Archivos:** public/offline.html, public/sw.js, public/js/conexion.js
- **Verificación:** Con el servidor apagado: catálogo, ficha cacheada y ficha no visitada funcionan
- **Commit:** 4518bc0

Fase 8 · Reseñas y Background Sync

API de reseñas idempotente en el servidor, cola local en IndexedDB y evento sync en el service worker que envía las pendientes al recuperar la red.

- **Archivos:** server.js, public/js/resenas.js, public/sw.js
- **Verificación:** Reseña escrita offline aparece «pendiente» y se publica sola al reconectar
- **Commit:** b57e283

Fase 9 · Ciclo de vida del SW

La versión nueva queda en espera hasta que el usuario pulsa «Actualizar» (skipWaiting); al activarse toma el control de las pestañas (clients.claim).

- **Archivos:** public/js/registro-sw.js, public/sw.js
- **Verificación:** Cambio de versión → banner → recarga con la versión nueva
- **Commit:** 5c7c538

Fase 10 · Notificaciones locales y push

Claves VAPID generadas automáticamente, suscripciones guardadas en el servidor, envío con web-push, recepción en el service worker y clic en la notificación que abre la app.

- **Archivos:** server/push.js, public/js/notificaciones.js, vista-avisos.js, public/sw.js
- **Verificación:** Push real entregado por WNS y guardado en el historial de avisos
- **Commit:** 0ebea3f

Fase 11 · Dispositivo e hidratación

Módulo de GPS (Haversine y rumbo), vibración y compartir; vista «Cerca de mí»; hidratación de la ficha SSR.

- **Archivos:** public/js/dispositivo.js, vista-cerca.js, ficha-ssr.js
- **Verificación:** Parada más cercana: Universidad a 176 m al noroeste; ficha SSR hidratada
- **Commit:** 68cdada

Fase 12 · Pruebas

Diez pruebas automáticas del servidor con node:test y plantilla del reporte de pruebas con 26 casos.

- **Archivos:** test/servidor.test.js, docs/PRUEBAS.md
- **Verificación:** 10 de 10 pruebas pasan
- **Commit:** 8d8abaf

Fase 13 · Publicación

Workflow de GitHub Actions que ejecuta las pruebas y publica public/ en GitHub Pages en cada push; guía de publicación en el README.

- **Archivos:** .github/workflows/pages.yml, README.md
- **Verificación:** Sitio publicado; simulación de subdirectorio estático verificada
- **Commit:** 61afe88

6. Mejoras posteriores

Después de completar las 13 fases se hicieron cuatro mejoras, cada una con su propio commit.

6.1 Mapa real

Se integró Leaflet con mapas de OpenStreetMap en tres pantallas: el catálogo (un pin por parada, sincronizado con los filtros), «Cerca de mí» (tu posición, el margen de error del GPS y una línea hasta la parada) y la ficha (mapa pequeño de ubicación). Leaflet se sirve desde la propia aplicación y las zonas visitadas se guardan para verse sin conexión, respetando la política de uso de OpenStreetMap, que no permite descargar zonas por adelantado.

6.2 Instalación más robusta

Se detectó que el botón «Instalar app» no hacía nada cuando el navegador no entregaba el aviso de instalación. Ahora el aviso se captura en cuanto carga la página y, si no existe, el botón muestra instrucciones según el navegador: Chrome o Edge, iPhone, Firefox, Samsung Internet o navegadores dentro de otras apps como WhatsApp.

6.3 Rediseño con identidad local

Se reemplazó el diseño genérico por una identidad propia de la Sierra Gorda:

- Paleta tomada de las fotografías del lugar: terracota y ocre de los portales y las misiones, verde de la sierra y fondo color cantera.
- Tipografías con carácter, incluidas en la app para verse sin conexión.
- Íconos SVG propios en lugar de emojis y una marca nueva: un arco de misión con un camino.
- Letreros de ruta con color fijo, como los del parabrisas de las combis, y un tablero de horarios inspirado en los andenes de una terminal.
- Fotos optimizadas a WebP en varios tamaños (de varios MB a entre 28 y 147 KB).

6.4 Reseñas reales con Supabase

Como GitHub Pages no tiene servidor, las reseñas se trasladaron a Supabase. Así se comparten entre todos los usuarios también en la versión publicada y llegan en tiempo real. Se mantuvo el principio de «guardar local primero, enviar después» y se agregaron reglas de seguridad, límite antispam y un botón para reportar comentarios inapropiados.

7. Pruebas

7.1 Herramientas

Herramienta	Uso
node:test (npm test)	Pruebas automáticas del servidor: API, SSR, reseñas, push y archivos PWA
Microsoft Edge + Chrome DevTools Protocol	Pruebas de extremo a extremo controladas por script: offline real, GPS simulado, push, actualización del SW, capturas
DevTools › Application	Revisar manifiesto, service worker, Cache Storage, IndexedDB y localStorage
DevTools › Network y Sensors	Simular sin conexión y coordenadas GPS
GitHub Actions	Ejecutar las pruebas antes de cada publicación

7.2 Pruebas automáticas del servidor

Caso	Qué se comprueba	Resultado
CP-01	/api/salud responde correctamente	Pasa
CP-02	El catálogo trae paradas con rutas, horarios y tarifa	Pasa
CP-03	La ficha SSR llega como HTML completo con datos para hidratar	Pasa
CP-04	Una parada inexistente devuelve 404	Pasa
CP-05	Una reseña publicada aparece en la API y en la ficha SSR	Pasa
CP-06	Reenviar la misma reseña no la duplica	Pasa
CP-07	Se rechazan reseñas inválidas y el HTML se escapa (prevención de XSS)	Pasa
CP-08	Clave pública VAPID y validación de suscripciones	Pasa
CP-09	Manifiesto, service worker con sus 6 eventos y offline.html	Pasa
CP-10	Todos los archivos precacheados por el SW existen	Pasa

Resultado: 10 de 10 pruebas pasan.

7.3 Pruebas en el navegador

Prueba	Resultado observado	Estado
Registro del SW y control de la página	activated; la página queda controlada en la primera visita	Pasa
Offline con el servidor apagado	8 paradas desde caché/IndexedDB; indicador «Sin conexión»	Pasa

Prueba	Resultado observado	Estado
Ficha no visitada sin conexión	Redirige a la ficha CSR con datos locales	Pasa
Reseña offline + Background Sync	Queda pendiente y se publica al reconectar	Pasa
Reseña en Supabase	La tabla responde y guarda la reseña	Pasa
Actualización del SW	Banner «Hay una nueva versión» → versión nueva activa	Pasa
Push real (VAPID)	Suscripción en WNS, envío 1/1, aviso guardado	Pasa
Cerca de mí	Parada Universidad a 176 m al noroeste, ~2 min	Pasa
Mapa	8 pines; 2 al filtrar combis; teselas guardadas en caché	Pasa
Instalabilidad en GitHub Pages	Edge: 0 errores de instalabilidad	Pasa
Splash del sistema, vibración y compartir	Requieren un teléfono Android real	Pendiente
Auditoría Lighthouse	Ejecutar en DevTools › Lighthouse	Pendiente

El detalle de los 26 casos y la plantilla para anotar evidencias está en docs/PRUEBAS.md.

8. Publicación

8.1 Repositorio y GitHub Pages

1. Se subió el repositorio a github.com/Kaidav54/pwaauto.
2. En Settings › Pages se eligió como origen «GitHub Actions».
3. El workflow pages.yml ejecuta las pruebas y publica la carpeta public/ en cada push a la rama main.
4. La aplicación queda disponible en kaidav54.github.io/pwaauto.

8.2 Qué funciona en GitHub Pages y qué necesita el servidor

Función	GitHub Pages	Servidor Node
Instalación, splash y manifiesto	Sí	Sí
App shell, offline y caché	Sí	Sí
Catálogo y mapa	Sí (data/paradas.json)	Sí (/api/paradas)
Ficha de parada	Sí, generada en el navegador	Sí, generada en el servidor + hidratación
Reseñas y Background Sync	Sí (Supabase, en tiempo real)	Sí (Supabase)
Cerca de mí, vibrar y compartir	Sí	Sí
Notificación local	Sí	Sí
Notificaciones push	No: enviarlas requiere un servidor	Sí

8.3 Cómo publicar una actualización

1. Hacer los cambios y subir la constante VERSION en public/sw.js (por ejemplo, de v9 a v10).
2. Ejecutar npm test.
3. Hacer commit y push a main; GitHub Actions publica la nueva versión.
4. Los usuarios ven el aviso «Hay una nueva versión» y al pulsar «Actualizar» se activa.

9. Capturas de pantalla

Capturas tomadas en un viewport de celular (390 × 844).

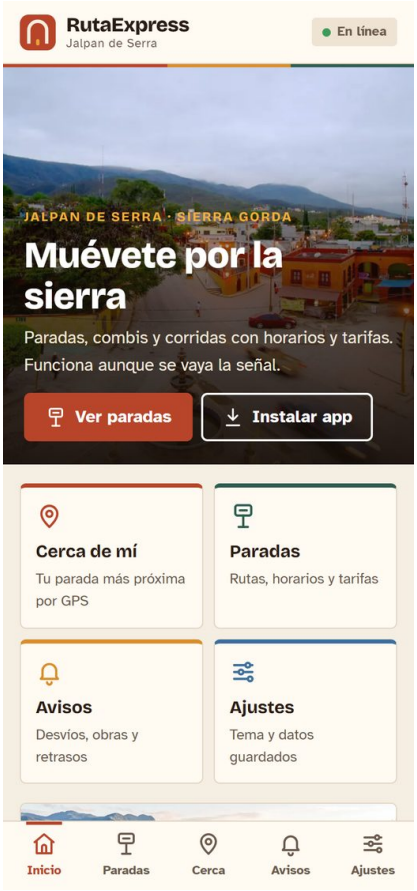


Figura 1. Inicio con la foto de Jalpan y el botón de instalación

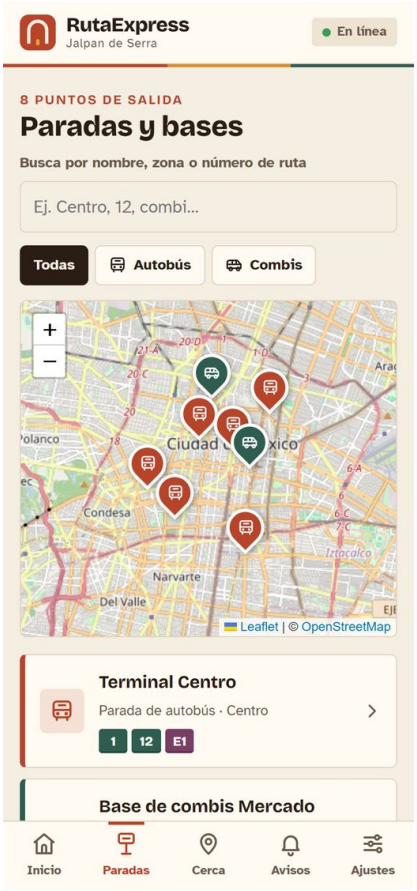


Figura 2. Catálogo con buscador, filtros y mapa



Figura 3. Ficha SSR hidratada con mapa de ubicación

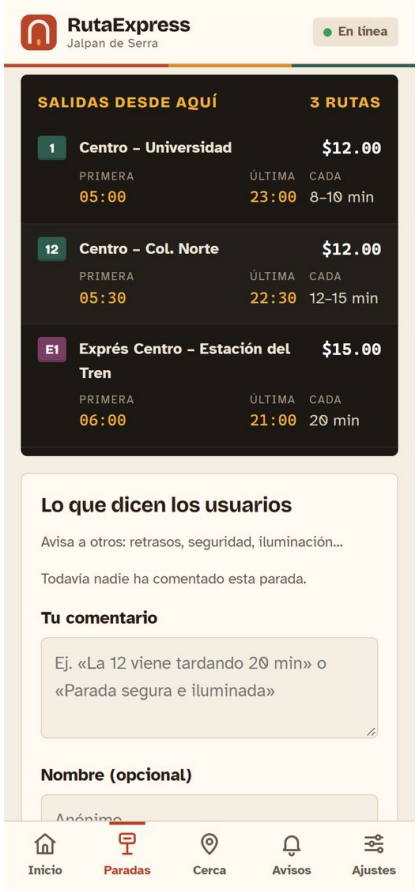


Figura 4. Tablero de salidas, horarios y tarifas

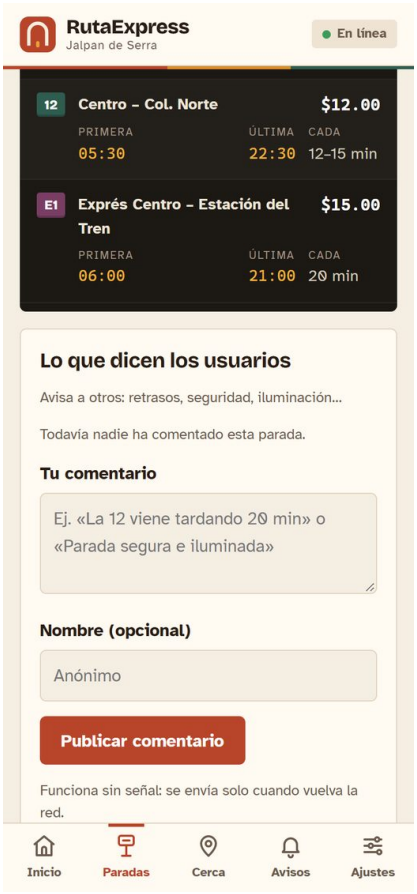


Figura 5. Comentarios de usuarios y formulario

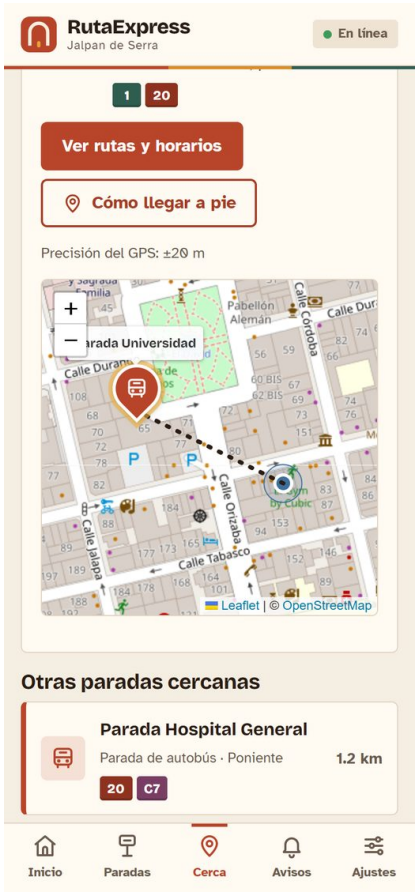


Figura 6. «Cerca de mí»: ruta a pie hasta la parada



Figura 7. Avisos push y panel de demostración



Figura 8. Ajustes: tema y datos guardados



Figura 9. Sin conexión: el catálogo sigue disponible



Figura 10. Tema oscuro

10. Limitaciones y trabajo futuro

- **Datos de ejemplo.** Las paradas del catálogo, sus horarios y tarifas son de ejemplo y sus coordenadas se ubican en la Ciudad de México. El siguiente paso es levantar en campo los datos reales de Jalpan de Serra (nombre, coordenadas, rutas, horarios y tarifas) y reemplazar public/data/paradas.json.
- **Push en la versión publicada.** GitHub Pages no puede enviar notificaciones. Para tenerlas en línea hay que desplegar el servidor Node en un servicio como Render o Railway.
- **Créditos de imágenes.** Falta confirmar el autor y la licencia de las fotografías del centro de Jalpan y de la misión, o sustituirlas por fotos propias. La imagen del autobús en la sierra es ilustrativa, generada con IA, y así se indica en la app.
- **Pruebas en dispositivo real.** Quedan pendientes el splash del sistema, la vibración, la hoja de compartir y la auditoría Lighthouse en un teléfono Android.
- **iPhone.** En iOS la instalación es manual (Compartir › Agregar a pantalla de inicio), no hay Background Sync y el push solo funciona con la app instalada.
- **Moderación.** Los comentarios se ocultan con 3 reportes; la moderación manual se hace con las consultas incluidas en docs/supabase.sql.

11. Conclusiones

Se construyó una Aplicación Web Progresiva completa siguiendo las 13 fases de la guía, con un commit por fase y verificación en cada paso. La aplicación cumple los cuatro resultados esperados: se instala, funciona sin conexión, sincroniza las reseñas al recuperar la red y recibe notificaciones push.

El proyecto mostró que el service worker es la pieza central de una PWA: además de la caché, coordina el envío en segundo plano, las notificaciones y las actualizaciones. También mostró la utilidad de combinar CSR y SSR con una plantilla compartida, y de diseñar «sin conexión primero» en un contexto real como la Sierra Gorda, donde la señal es intermitente.

Las mejoras posteriores (mapa real, rediseño con identidad local y reseñas compartidas en tiempo real con Supabase) acercan la aplicación a un producto utilizable. Con los datos reales de las paradas de Jalpan de Serra quedaría lista para usuarios.

Anexo A. Enlaces y comandos

Recurso	Dirección
Repositorio	https://github.com/Kaidav54/pwaauto
Aplicación publicada	https://kaidav54.github.io/pwaauto/
Script de base de datos	docs/supabase.sql
Reporte de pruebas	docs/PRUEBAS.md

Comando	Qué hace
npm install	Instala las dependencias
npm start	Inicia el servidor en http://localhost:3000
npm test	Ejecuta las pruebas automáticas
npm run iconos	Regenera los íconos de la app
npm run imagenes	Optimiza las fotos de assets/jalpan a WebP
npm run vapid	Genera las claves VAPID para push

Anexo B. Historial de commits

Commit	Descripción
35e0061	Fase 0: entorno, dependencias y servidor mínimo
eaa0e32	Fase 1: app shell (index.html) y hoja de estilos
56eb2dd	Fase 2: manifiesto e íconos generados
0f8daf5	Fase 3: registro del service worker y sw.js con install/activate/fetch
c350c81	Fase 4: catálogo remoto, enrutador CSR y ficha SSR
60ef834	Fase 5: splash dentro de la app y Home con botón de instalación
4b5e2bc	Fase 6: IndexedDB, preferencias en localStorage y respaldo local del catálogo
4518bc0	Fase 7: offline.html, estrategias de caché e indicador de conexión
b57e283	Fase 8: API de reseñas, cola local y Background Sync
5c7c538	Fase 9: ciclo de vida del SW, skipWaiting/clients.claim y aviso de nueva versión
0ebea3f	Fase 10: notificaciones locales y push con VAPID
68cdada	Fase 11: módulo de dispositivo, vista Cerca de mí e hidratación de la ficha SSR

Commit	Descripción
8d8abaf	Fase 12: pruebas automáticas del servidor y plantilla del reporte de pruebas
61afe88	Fase 13: workflow de GitHub Pages y guía de publicación
3509d6f	Docs: enlaces del repositorio y de GitHub Pages
9be321b	Mapa real con Leaflet/OpenStreetMap e instalación con instrucciones de respaldo
3ddf365	Rediseño con identidad de Jalpan de Serra
42161cf	Reseñas reales compartidas con Supabase